

Edge Addition Planarity Suite and Generalized Graph Library

John M. Boyer¹ and Wanda B. K. Boyer¹

¹ Independent Researcher, Canada ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

A *graph* is a mathematical structure comprising a set V of vertices and a set E of edges, with each edge e corresponding to a pair of vertices called the *endpoints* of e . A graph is *planar* if its vertices can be placed in distinct locations on a plane surface and its edges can be drawn on the plane without the edges intersecting, except at their common vertex endpoints. For a planar graph, a planarity algorithm typically outputs a combinatorial data structure that validation code can use to certify the planarity of the input graph. A planar graph drawing is typically produced by a separate algorithm. On the other hand, if an input graph is not planar, then a planarity algorithm typically outputs a minimal subgraph of the input graph that obstructs planarity. Validation code can use a minimal planarity-obstructing subgraph to certify the non-planarity of an input graph. Moreover, a minimal subgraph obstructing planarity can be used to help decide how to amend an input graph to *planarize* it.

Graphs are used to model a very wide array of real-world problems in which there are objects and relationships between the objects. In artificial intelligence, graphs are used to help with reasoning tasks, such as about the relationship pathways between persons of interest in law enforcement or between genes, tissues, diseases, and medications in bioinformatics research. In Physics, graphs and planarity are used to help compute material phase transitions, particle interactions, and electromagnetic duality. Similarly, in chemistry, graphs may be used to represent atoms and their valence bonds in molecules. Planarity testing can help determine feasible molecular arrangements because the molecular graphs for many compounds of interest must be planar. In electronics, a graph may be used to represent the electrical components and wiring in a circuit, such as transistors and etchings on a silicon wafer. The graph of a circuit must be planar or planarized to eliminate short circuits.

The open source software described in this paper provides a highly performant generalized graph library that also implements state-of-the-art algorithms for planarity-related problems. The generalized graph library and planarity-related algorithms are available to C/C++ and Python graph application developers.

Statement of need

The Edge Addition Planarity Suite is an open source software package that was originally developed to implement a new planarity algorithm in (Boyer & Myrvold, 2004). The Edge Addition Planarity Algorithm is currently regarded as one of two state-of-the-art planarity algorithms due to its conceptual simplicity as well as its speed of execution (Contributors to Wikipedia, 2026). The software package is available from the [edge-addition-planarity-suite GitHub repository](#) under a 3-clause BSD license.

To enable development of the Edge Addition Planarity Algorithm, as well as the other planarity-related algorithms in (Boyer, 2006) and (Boyer, 2012), it was necessary to develop an extensible,

41 generalized graph library. This generalized graph library provides a high-performance solution
42 for the graph algorithm representation and processing needs of a very wide array of graph
43 applications.

44 In addition to enabling graph application development in C/C++, the popularity of Python as
45 a scientific application development language has prompted the need for making this
46 graph library and the algorithms it implements available to Python developers. The source code
47 may be accessed by the [planarity Github repository](#), and the [planarity Python package](#) may
48 be installed via the Python Package Index (PyPI) by simply running `pip install planarity`.

49 State of the field

50 Several large scientific software packages offer a graph planarity algorithm, which is a complex
51 graph algorithm suitable for benchmarking graph libraries. Mathematica and Wolfram Alpha
52 include planarity algorithms, but these packages are not meant for high performance graph
53 algorithms, so the Library for Efficient Data Types and Algorithms ([LEDA](#)) is significantly
54 faster than they are because it is an industrial library designed for computational efficiency. Yet,
55 in ([Boyer et al., 2004](#)), all planarity algorithms implemented in LEDA were shown to be several
56 times slower than the Edge Addition Planarity Algorithm implementation. LEDA also has the
57 disadvantage of being C++ only and has a proprietary license. The only graph library shown
58 (in ([Boyer et al., 2004](#))) to have similar performance to the Edge Addition Planarity Suite is the
59 Public Implementation of a Graph Algorithm Library and Editor ([PIGALE](#)). However, PIGALE
60 is also C++ only, has a GPLv2 license, and lacks advanced algorithms for homeomorphic
61 subgraph search. For example, torus embedding algorithms and torus obstruction isolation
62 algorithms are able to operate differently based on the result of a search for a subgraph
63 homeomorphic to $K_{3,3}$ ([Myrvold & Kocay, 2011](#); [Myrvold & Woodcock, 2018](#)).

64 Software design

65 The generalized graph library in the Edge Addition Planarity Suite has a carefully curated API
66 and an object-oriented architecture. The base **Graph** class structure includes

- 67 ■ base graph, vertex, and edge structure definitions and getter/setter methods;
- 68 ■ graph, vertex and edge allocation, deallocation, and reset methods;
- 69 ■ methods for iterating through all vertices, edges, and edge adjacency lists;
- 70 ■ graph readers and writers for several formats; and
- 71 ■ a dynamic subclassing/extension mechanisms and virtual function table.

72 The base Graph is extendable with utility methods for exploring a graph, labelling vertices and
73 edges according to a depth-first search (DFS), and for managing connectivity and biconnectivity
74 information. The vertex and edge structures are extended to enable storage of the information
75 generated by the utility methods.

76 A Graph structure extended with the DFS-related utilities is further extendable to a **Planarity**
77 **Graph**, which also further extends the vertex and edge structures. The main additional method
78 provided by a Planarity Graph is `gp_Embed()`, which takes a Planarity Graph as input and
79 outputs either a combinatorial planar embedding or a minimal planarity-obstructing subgraph.

80 Given a Planarity Graph, a number of subclass extensions are available for planarity-related
81 problems. One extension solves outerplanarity. Another implements the planar graph drawing
82 algorithm in ([Boyer, 2006](#)). Three other extensions provide Graph subclasses that solve the
83 homeomorphic subgraph search algorithms appearing in ([Boyer, 2012](#)). All of these extensions
84 further extend the vertex and edge structures and overload the `gp_Embed()` function.

85 In addition to being available to C and C++ developers, the APIs of the Edge Addition
86 Planarity Suite and Generalized Graph Library are also made available via the planarity

87 Python package with the source code made available in the [planarity Github repository](#).
88 Cython is used to preserve high performance while also broadening availability to the Python
89 developer community.

90 A final aspect of this open source software project is our emphasis on software reliability.
91 We perform validation code on `gp_Embed()` and its five overloads on billions of randomly
92 generated graphs up to 10 million vertices and 30 million edges. The GitHub actions that
93 run on every pull request include sanitizer tests on all graphs on 8 vertices. Finally, the [Edge
94 Addition Planarity Suite Testing GitHub repository](#) provides our multithreaded Python code for
95 validating `gp_Embed()` and its five overloads on the more than one billion graphs having 11 or
96 fewer vertices, as generated by the geng tool in Nauty and Traces (McKay & Adolfo, 2025).

97 Research impact statement

98 The journal publication of the Edge Addition Planarity Algorithm, (Boyer & Myrvold, 2004),
99 currently has over 360 scholarly citations according to [Google Scholar](#). The algorithm has been
100 implemented in several large open source projects, including [Boost](#), [Magma](#), [nauty](#) and [Traces](#)
101 (McKay & Adolfo, 2025), and the [Open Graph Drawing Framework](#) ([web archive](#)). The source
102 code from the Edge Addition Planarity Suite repository has been directly included in several
103 large open source projects, including [SageMath](#), [Digraphs](#), and over 80 Linux platforms releases
104 including for Debian, Devuan, Kali, openSUSE, Raspbian, Ubuntu, Alpine, ALT, Fedora,
105 FreeBSD, Gentoo, Manjaro, MSYS2, and Parabola, according to [repology/edge-addition-
106 planarity-suite](#) ([web archive](#)) and [repology/planarity](#) ([web archive](#)).

107 The Python package named `planarity` was originally developed in 2013 by Aric Hagberg as a
108 way to get the planarity testing and drawing algorithms from (Boyer & Myrvold, 2004) and
109 (Boyer, 2006) to work with graphs from NetworkX (Hagberg et al., 2008). Recently, Hagberg
110 transferred the [planarity Github repository](#) to the [GitHub Graph Algorithms Organization](#)
111 and the [planarity Python package](#) to the [PyPI Graph Algorithms Organization](#), which are
112 both maintained by the authors. On August 1, 2026, the [Top PyPI Packages](#) website indicated
113 that the `planarity` Python package had over 700,000 downloads for the preceding month and
114 was ranked in the top 1% of PyPI package downloads (ranked 5262 out of [864,036 projects](#)).
115 According to [pepy.tech](#) ([web archive](#)) the `planarity` Python package has been downloaded
116 over 2 million times in total. The `planarity` Python package has now been included as a
117 standard Python package in several versions of Linux including Debian, Devuan, Kali, Raspbian,
118 and Ubuntu, according to [repology/python:planarity](#) ([web archive](#)).

119 AI usage disclosure

120 No generative AI tools were used in the development of this software, the writing of this
121 manuscript, nor the preparation of supporting materials.

122 References

- 123 Boyer, J. M. (2006). A new method for efficiently generating planar graph visibility
124 representations. In P. Eades & P. Healy (Eds.), *Proceedings of the 13th international
125 symposium on graph drawing 2005* (Vol. 3843, pp. 508–511). Springer-Verlag.
126 https://doi.org/10.1007/11618058_47
- 127 Boyer, J. M. (2012). Subgraph homeomorphism via the edge addition planarity algorithm.
128 *Journal of Graph Algorithms and Applications*, 16(2), 381–410. [https://doi.org/10.7155/
129 jgaa.00268](https://doi.org/10.7155/jgaa.00268)
- 130 Boyer, J. M., Cortese, P. F., Patrignani, M., & Di Battista, G. (2004). Stop minding your
131 P's and Q's: Implementing a fast and simple DFS-based planarity testing and embedding

- 132 algorithm. In G. Liotta (Ed.), *Proceedings of the 11th international symposium on graph*
133 *drawing 2003* (Vol. 2912, pp. 25–36). Springer-Verlag. [https://doi.org/10.1007/978-3-](https://doi.org/10.1007/978-3-540-24595-7_3)
134 [540-24595-7_3](https://doi.org/10.1007/978-3-540-24595-7_3)
- 135 Boyer, J. M., & Myrvold, W. J. (2004). On the cutting edge: Simplified $O(n)$ planarity
136 by edge addition. *Journal of Graph Algorithms and Applications*, 8(3), 241–273. <https://doi.org/10.7155/jgaa.00091>
137
- 138 Contributors to Wikipedia. (2026). Planarity testing. In *Wikipedia, the free encyclopedia*.
139 Wikipedia. [https://en.wikipedia.org/w/index.php?title=Planarity_testing&oldid=](https://en.wikipedia.org/w/index.php?title=Planarity_testing&oldid=1336627782)
140 [1336627782](https://en.wikipedia.org/w/index.php?title=Planarity_testing&oldid=1336627782)
- 141 Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics,
142 and function using NetworkX. *Python in Science Conference*. [https://doi.org/10.25080/](https://doi.org/10.25080/TCWV9851)
143 [TCWV9851](https://doi.org/10.25080/TCWV9851)
- 144 McKay, B. D., & Adolfo, P. (2025). *Nauty and traces user's guide (version 2.9.0)*. [https://](https://pallini.di.uniroma1.it/nug29.pdf)
145 pallini.di.uniroma1.it/nug29.pdf
- 146 Myrvold, W., & Kocay, W. (2011). Errors in graph embedding algorithms. *Journal of Computer*
147 *and System Sciences*, 77(2), 430–438. <https://doi.org/10.1016/j.jcss.2010.06.002>
- 148 Myrvold, W., & Woodcock, J. (2018). A large set of torus obstructions and how they were
149 discovered. *The Electronic Journal of Combinatorics*, 25(1), 1–17. [https://doi.org/10.](https://doi.org/10.37236/3797)
150 [37236/3797](https://doi.org/10.37236/3797)

DRAFT